CO1 – AT1: - Analytical Problem Solving

Name: - D. Sri Anjaneya **Reg. No.: -** 192425219

Course Name: - Deep Learning **Course Code: -** MLA0409

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:
Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer:

Effect of a High Initial Learning Rate Followed by Learning Rate Decay:

Gradient descent updates model parameters using the learning rate (η). The learning rate determines the size of each step taken toward the minimum of the loss function.

1. Effect on Convergence:

Using a high initial learning rate allows the algorithm to take large steps during the early stages of training. This helps the model:

- Reach the region near the global minimum quickly.
- Reduce training time by making faster progress.
- Avoid spending too much time in areas with large gradients.

As training continues, the learning rate is gradually decreased (learning rate decay). Smaller steps then allow the algorithm to:

- Fine-tune the model parameters.
- Reduce oscillations around the minimum.
- Achieve more stable and accurate convergence.

Thus, the combination provides fast initial learning followed by precise optimization.

2. When Does This Strategy Outperform a Constant Learning Rate:

A high initial learning rate with decay performs better than a constant learning rate when:

- The loss function is convex, so there is a single global minimum.
- The initial learning rate is large enough to speed up progress but not so large that it causes divergence.
- The learning rate decays at an appropriate rate.
- The optimization requires both fast exploration and accurate final convergence.

Compared with a constant learning rate:

- A small constant learning rate converges slowly.
- A large constant learning rate may never settle near the minimum because it keeps taking large steps.
- A decaying learning rate combines the advantages of both.

3. Possible Risks:

This strategy also has some risks:

- Learning rate too high initially
  - The algorithm may overshoot the global minimum.
  - Loss may oscillate or even diverge.
  - Training may become unstable.
- Learning rate decreases too quickly
  - Updates become very small too early.
  - The model may stop improving before reaching the optimum.
- Learning rate decreases too slowly
  - The model may continue oscillating around the minimum.
  - Convergence becomes slower than expected.

4. Justification:

The strategy works because optimization has two phases:

- Early stage: Large updates rapidly reduce the loss and move the parameters close to the optimum.
- Later stage: Small updates allow careful adjustments, improving stability and accuracy.

This balance between speed and precision often leads to faster and better convergence than using a fixed learning rate throughout training.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:
Derive the Maximum Likelihood Estimator for μ, and analytically explain how the estimate changes when:

- The dataset contains outliers
- The sample size increases
  What does this imply about the robustness of MLE?

Answer:

Suppose the dataset $x_1, x_2, \dots, x_n$ is independently drawn from a Gaussian distribution:

$$X_i \sim \mathcal{N}(\mu, \sigma^2)$$

where:

- Unknown: Mean μ
- Known: Variance σ^2

1. Derivation of the MLE for μ:

The probability density function of a Gaussian distribution is:

$$f(x_i; \mu) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2} \right)$$

The likelihood function is:

$$L(\mu) = \prod_{i=1}^{n} f(x_i; \mu)$$

Taking the logarithm gives the log-likelihood:

$$\ln L(\mu) = -\frac{n}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^{n} (x_i - \mu)^2$$

Differentiate with respect to μ:

$$\frac{d}{d\mu} \ln L(\mu) = \frac{1}{\sigma^2} \sum_{i=1}^{n} (x_i - \mu)$$

Set the derivative equal to zero:

$$\sum_{i=1}^{n}(x_i - \mu) = 0$$

$$\sum_{i=1}^{n} x_i - n\mu = 0$$

Hence,

$$\boxed{\hat{\mu}_{MLE} = \frac{1}{n}\sum_{i=1}^{n} x_i}$$

Thus, the Maximum Likelihood Estimator of the mean is the sample mean.

2. Effect of Outliers:

Since the MLE is the sample mean, every observation contributes equally.

Suppose the data are:

$$5, 6, 7, 8, 9$$

The estimated mean is:

$$\hat{\mu} = 7$$

If one outlier is added:

$$5, 6, 7, 8, 100$$

The new estimate becomes:

$$\hat{\mu} = 25.2$$

Although only one value changed, the estimate shifts dramatically.

Reason:

- The mean gives equal weight to every data point.
- Extremely large or small observations pull the estimate toward themselves.
- Therefore, the MLE for the Gaussian mean is highly sensitive to outliers.

3. Effect of Increasing Sample Size:

As the number of observations $n$increases,

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

becomes a more accurate estimate of the true mean.

According to the Law of Large Numbers:

- The sample mean converges to the true mean.
- Random fluctuations decrease.
- The variance of the estimator becomes

$$Var(\hat{\mu}) = \frac{\sigma^2}{n}$$

Hence,

- Larger sample size → Smaller variance
- Larger sample size → More stable estimate
- Larger sample size → Higher accuracy

4. Implication for the Robustness of MLE:

The robustness of an estimator refers to its ability to resist the influence of abnormal observations.

For the Gaussian MLE:

Advantages

- Efficient when data truly follow a normal distribution.
- Unbiased estimator of the mean.
- Consistent estimator (approaches the true mean as sample size increases).
- Variance decreases with increasing sample size.

Disadvantages

- Highly sensitive to outliers.
- A few extreme observations can significantly change the estimate.
- Not robust when the dataset contains noisy or contaminated data.

In such cases, robust estimators such as the median or trimmed mean are often preferred.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:
Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer:

Given:

- Features: 10,000
- Training samples: 500

This is a high-dimensional, low-sample-size (HDLSS) problem, where the number of features is much larger than the number of training examples.

1. Challenges in Training the Model:

a) Curse of Dimensionality

- With 10,000 features, data becomes sparse.
- The model struggles to identify meaningful patterns.
- Distance-based algorithms become less effective.

b) Overfitting

- The model may memorize the training data instead of learning general patterns.
- High training accuracy but poor performance on unseen test data.

c) High Computational Cost

- More features require more memory and longer training time.
- Model optimization becomes computationally expensive.

d) Irrelevant and Redundant Features

- Many features may contain little or no useful information.
- Noise can reduce model accuracy.

e) Small Sample Size

- Only 500 training samples are insufficient to reliably estimate the importance of 10,000 features.
- The model has a high risk of learning random noise.

2. Proposed Machine Learning Pipeline:

Step 1: Data Preprocessing

- Handle missing values.
- Normalize or standardize the features.
- Remove duplicate samples.

Justification:

- Ensures all features are on a comparable scale.
- Improves optimization and model stability.

Step 2: Feature Selection

Use techniques such as:

- Variance Threshold
- Chi-Square Test
- Mutual Information
- L1 (Lasso) Feature Selection

Reduce features from 10,000 → 500–1000 important features.

Justification:

- Removes irrelevant features.
- Reduces noise.
- Decreases computational cost.
- Improves model interpretability.

Step 3: Dimensionality Reduction

Apply Principal Component Analysis (PCA).

Example:

- 1000 selected features
- Reduce to 100 principal components

Justification:

- Compresses correlated features.
- Retains most of the data variance.
- Produces a simpler feature representation.

Step 4: Train a Regularized Classifier

Suitable algorithms include:

- Logistic Regression with L2 Regularization
- Support Vector Machine (Linear SVM)
- Lasso (L1) Logistic Regression

Justification:

- Regularization penalizes overly complex models.
- Prevents excessively large model weights.
- Improves generalization to unseen data.

Step 5: Cross-Validation

Use k-Fold Cross-Validation (e.g., 5-fold or 10-fold).

Justification:

- Makes efficient use of the limited data.
- Provides a more reliable estimate of model performance.
- Helps select optimal hyperparameters.

Step 6: Model Evaluation

Evaluate using:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC (if appropriate)

Justification:

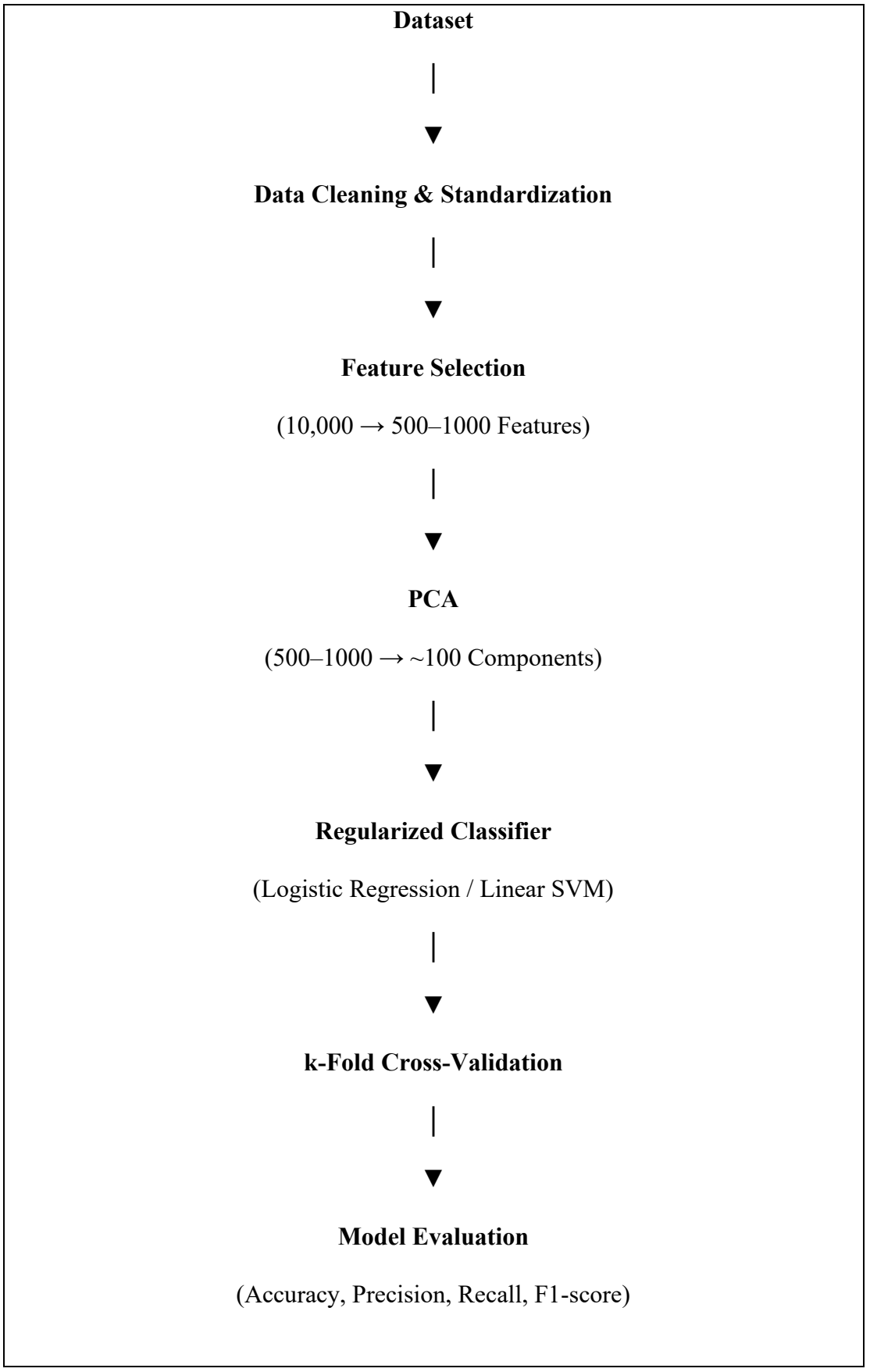
- Multiple metrics provide a balanced assessment, especially if classes are imbalanced.

3. How This Pipeline Mitigates Overfitting

Technique	How It Prevents Overfitting
Feature Selection	Removes noisy and irrelevant features.
PCA	Reduces dimensionality while preserving important information.
Regularization (L1/L2)	Penalizes complex models and large coefficients.
Cross-Validation	Ensures the model generalizes well to unseen data.
Simpler Classifier	Less likely to memorize training data than highly complex models.

Together, these steps reduce model complexity and improve generalization.

4. Recommended Algorithm Pipeline:



4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:
Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer:

Given:

A Multilayer Perceptron (MLP) has one hidden layer and uses the sigmoid activation function.

1. Why Does Adding More Hidden Neurons Increase the Representational Power?

Each hidden neuron learns a different feature or pattern from the input data.

The output of a sigmoid neuron is

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- With few hidden neurons, the network can learn only simple patterns.
- With more hidden neurons, the network can learn more complex relationships between inputs and outputs.

For example:

- 2 hidden neurons → Learn simple decision boundaries.
- 10 hidden neurons → Learn more detailed and complex patterns.

Thus, increasing the number of hidden neurons increases the network's ability to represent complex functions.

2. Why Doesn't Adding More Hidden Neurons Always Improve Performance?

Although more neurons increase learning capacity, they also increase the number of parameters (weights and biases).

More neurons

$$\Rightarrow \text{More Parameters}$$

If the training data are limited, the network may start memorizing the training data instead of learning general patterns.

This leads to overfitting.

3. Overfitting

When too many hidden neurons are used,

- Training Accuracy → Very High
- Test Accuracy → Low

The model performs well on training data but poorly on new data because it has learned the noise instead of the actual pattern.

4. Generalization

Generalization means the model performs well on unseen data.

A network with the right number of hidden neurons learns the important patterns without memorizing the training data.

Therefore,

- Moderate number of neurons → Better generalization.
- Too many neurons → Poor generalization due to overfitting.

5. Analytical Reasoning

Suppose:

- Model A has 5 hidden neurons.
- Model B has 100 hidden neurons.

Then,

$$100 > 5$$

Model B has a higher learning capacity and can represent more complex functions.

However, if the dataset is small,

$$\text{Model Complexity} \uparrow \Rightarrow \text{Overfitting} \uparrow$$

Hence,

$$\text{Generalization} \downarrow$$

So, increasing hidden neurons improves representational power, but beyond a certain point it reduces the model's ability to perform well on unseen data.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:
Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

Answer:

Given:

A Deep Neural Network (DNN) is trained using Stochastic Gradient Descent (SGD) on a high-dimensional dataset.

1. Impact of Curse of Dimensionality on Gradient Updates (Backpropagation)

In backpropagation, weights are updated as

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

where:

- w= weight
- η= learning rate
- L= loss function

In a high-dimensional dataset, there are many features and weights.

Result:

- More gradients need to be calculated.
- Some gradients become very small or noisy.
- Weight updates become less effective.

Therefore, backpropagation becomes slower and less stable.

2. Impact on Convergence Speed of SGD:

SGD updates weights using one training sample at a time.

In high dimensions:

- There are many parameters to update.
- SGD needs more iterations to find the minimum loss.

Therefore,

$$\text{High Dimensions} \rightarrow \text{More Parameters} \rightarrow \text{Slower Convergence}$$

Training takes longer because SGD must optimize many weights.

3. Impact on Model Generalization:

When the number of features is very high,

$$\text{Features} \gg \text{Training Samples}$$

the model may learn noise instead of useful patterns.

Result:

- Training Accuracy → High
- Test Accuracy → Low

This is called overfitting, which reduces the model's ability to generalize to new data.

4. Techniques to Overcome These Issues:

Technique 1: Dimensionality Reduction (PCA)

Reduce the number of features.

Example:

$$10{,}000 \rightarrow 200 \text{ features}$$

Justification

- Removes unnecessary information.
- Reduces the number of weights.
- Faster gradient updates.
- SGD converges more quickly.
- Less chance of overfitting.

Technique 2: Regularization (L2)

Regularization adds a penalty to the loss function.

$$\text{Loss} + \lambda \, || \, w \, ||^2$$

Justification

- Prevents very large weights.
- Reduces model complexity.
- Helps the model learn general patterns instead of memorizing the training data.
- Improves generalization.

5. Analytical Summary:

High-dimensional data causes:

- Gradient updates → Slower because many weights must be updated.
- SGD convergence → Slower due to many parameters.
- Generalization → Poor because the model may overfit the training data.

By using PCA and L2 Regularization:

- Number of features decreases.
- Number of weights decreases.
- Training becomes faster.
- Overfitting reduces.
- Generalization improves.